SD and microSD on FPGA

SPI Initialization, Sector Reading, Verilog, SystemVerilog, VHDL, and Verification

Sipeed FPGA - Educational and Research Series

24 October 2026 | Edition 0.1 | Teaching manuscript

Contents

1 Scope and proof levels **2**

2 SD, microSD and SPI: the actual stack **2**
2.1 Tang Nano 9K board signals 3

3 SPI mode, capacity families, and address units **3**

4 Power-up and the command sequence **4**
4.1 Six-byte SPI command frame 4
4.2 What comes back from CMD17? 5

5 FPGA architecture: state machines and RAM, not software **5**

6 Verilog: the real reference design **5**
6.1 Data CRC16: checking the payload rather than counting bytes 6

7 SystemVerilog: a portable-controller design sketch **7**

8 VHDL: the same protocol in a typed FSM **7**

9 Testbenches: a simulated card and negative cases **8**
9.1 SystemVerilog testbench outline 9
9.2 VHDL testbench outline 9

10 External scripts, synthesis, and SRAM-only loading **9**
10.1 Manually program *volatile SRAM* after checking reports 10
10.2 Error codes specific to this Verilog design 10

11 From a first sector to a filesystem: research program **11**
11.1 Physical experiment record template 11

12 References, code, and video demonstration **12**

Demonstrated result. A Tang Nano 9K FPGA directly initialized a **physical microSDHC card** in SPI mode and read the first 512-byte sector with **CMD17**, storing the bytes in FPGA block RAM and checking CRC16. No PicoRV32/RISC-V CPU was involved. The operator observed `SD PASS SIG=55AA` twice over UART. This is **not** FAT32, directory listing, file access, or an SD write demonstration. The independently published

source snapshot has a successful RTL CI gate, but **its newly generated bitstream has not received separate board acceptance**.

Reproducible SD-01 source: https://github.com/AIDevelopersMonster/Sipeed-FPGA/tree/main/boards/tang-nano-9k/experiments/sd-sector0

Demonstration video provided by the author: https://youtu.be/ybEUMzt7Gqg

Fixed code revision: 5b4ba625. **SPI-01 companion:** https://github.com/AIDevelopersMonster/Sipeed-FPGA/tree/main/examples/peripherals/spi/01-mode0-byte. **DOI:** pending author-supplied publication identifier.

1 Scope and proof levels

The independent SD-01 laboratory is an educational and research artifact in its own right, rather than merely a temporary PiKoSoL development checkpoint. The initial question is deliberately narrow: **can a standalone FPGA, using only synchronous digital logic, initialize an SDHC card and read a sector correctly?** More ambitious work can compare multiple architectures, including hardware FSMs, CPU-based drivers and hybrid controllers.

Stage	Actual capability	Evidence / status
SPI-01	Mode 0 loopback byte transfer	RTL PASS
Original SD-01	CMD0, CMD8, CMD55, ACMD41, CMD58, CMD17, LBA0, CRC16	Operator-observed BOARD PASS
Published SD-01 snapshot	Independent RTL, testbench, constraints, Gowin build	CI RTL PASS; independent board retest not claimed
SD-02	Arbitrary LBA and repeated sector reads	Research task, not SD-01
SD-03/04	Partition structures, FAT32, `DIR` and TYPE	Not implemented in SD-01
Card writes	CMD24/CMD25, data integrity, persistence	Entirely out of scope

Language status: the reference RTL is implemented in **Verilog**. The SystemVerilog and VHDL snippets below are architecture/verification sketches, **not complete or board-tested SDHC cores**.

2 SD, microSD and SPI: the actual stack

A microSD card is not raw NAND exposed directly to an FPGA. The card includes its own embedded flash controller that handles the underlying storage medium and presents a command/block interface. Our FPGA communicates through the card's **SPI mode**, not the native 1-bit or 4-bit SD host interface.

On Tang Nano 9K the programmed FPGA generates SCK, MOSI and CS_n and samples MISO

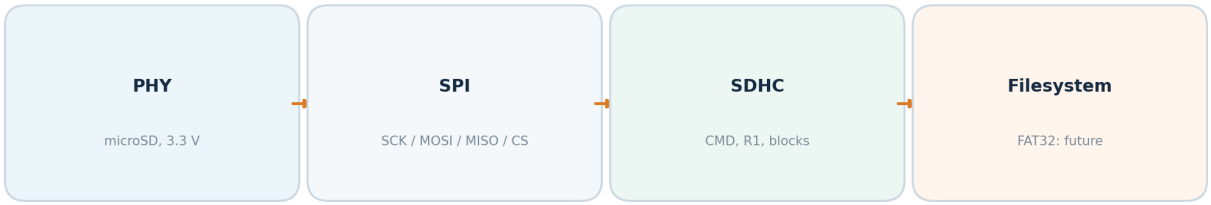


Figure 1: Hardware and software layers of the microSD design

directly. A separate Verilog UART transmitter reports the result to the computer. The FPGA design includes **no RISC-V CPU and no firmware executing instructions**.

2.1 Tang Nano 9K board signals

Function	FPGA pin	Electrical / timing note
Reference clock	52	27 MHz clock input
Reset	3	Active low; physical button S2 on tested board
SD SCK	36	3.3 V output
SD MOSI	37	3.3 V output
SD CS_n	38	3.3 V output
SD MISO	39	3.3 V input
UART TX	17	115200 baud, 8N1
LEDs	10/11/13/14/15/16	Active-low indicators

These values are **project pin constraints** for the specific Tang Nano 9K implementation; never reuse them blindly for Tang Primer 20K/25K, another PCB revision, or an unrelated FPGA. The actual electrical bank setup and timing reports must be inspected on every target.

3 SPI mode, capacity families, and address units

For an SDHC/SDXC card in SPI mode, **CMD17's argument is a logical block address (LBA)**; the transfer is a 512-byte block. Older SDSC devices use **byte addressing**, with separate initialization and block-size considerations. SD-01 deliberately **rejects unsupported SDSC behavior** rather than risk reporting a misleading successful read.

Two separate layers must not be confused:

- **SDHC block access:** initialize a card and read a numbered 512-byte sector.
- **FAT32 filesystem:** interpret partition tables or boot sectors, allocation tables, directories and file clusters stored in sectors.

The 0x55AA value in the final two sector bytes is common in MBR and boot-sector structures. By itself, **it does not prove the presence of FAT32, a valid partition table, or any readable file**. SD-01 prints the bytes as an observable diagnostic signature.

4 Power-up and the command sequence

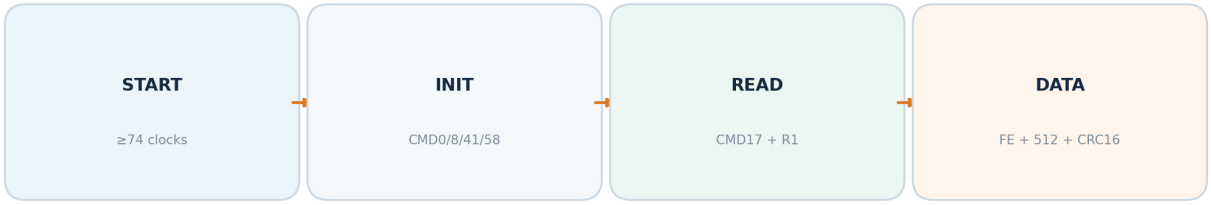


Figure 2: Steps from initialization to one sector and CRC16

With power stable, an SPI-mode SD card must receive at least **74 initial clock cycles** with the device not selected (MOSI held high). The controller in this project sends an initial run of idle clocks, then proceeds through the following sequence:

Command	Purpose	Expected check
CMD0	GO_IDLE_STATE; enter SPI idle	R1 normally reports idle: 0x01
CMD8	SEND_IF_COND; voltage and test pattern	R7 echo of 0x1AA
CMD55	APP_CMD prefix	Correct R1 response
ACMD41	Initialization until card becomes ready	Retry with timeout until R1 is 0x00
CMD58	READ_OCR	OCR; identify high-capacity mode / CCS
CMD17	READ_SINGLE_BLOCK	R1 0x00, 0xFE token, 512 bytes, CRC16

This is the sequence supported by the **published Verilog reference**. Operations such as CID/CSD (CMD9/CMD10), multi-block reading (CMD18) and writing (CMD24) are **not** part of the SD-01 artifact.

4.1 Six-byte SPI command frame

```
[0x40 | CMD] [ARG31:24] [ARG23:16] [ARG15:8] [ARG7:0] [CRC7 + 1]
    1 byte      1 byte      1 byte      1 byte    1 byte      1 byte
```

Concrete examples: CMD0 starts with 0x40, carries a zero argument and terminates with 0x95; CMD8 starts with 0x48, uses argument 0x000001AA and CRC byte 0x87. The SD-01 source uses fixed valid CRC values for these essential early commands and a simple trailing CRC byte for the remaining commands.

R1 is a one-byte command status; 0x00 means no R1 error bits are asserted. **R7** (CMD8) and **R3** (CMD58) include four additional bytes. After a successful CMD17 R1 response, the host must still wait for a **separate data-start token** 0xFE. Successful command reception is not the same as data arrival.

4.2 What comes back from CMD17?

```
CMD17 --> R1=00 --> [wait bytes FF ...] --> FE --> DATA[0..511] --> CRC_HI
 ↪  --> CRC_LO
                                        token      512 bytes      16-bit CRC
```

The logic collects the payload in sector[0:511], checks CRC16, and then requests bytes 510 and 511 through a separate synchronous RAM read interface. The UART message is assembled only after the sector transaction has finished.

5 FPGA architecture: state machines and RAM, not software

The design contains distinct cooperating hardware blocks:

1. **SPI byte master.** Serializes one byte into eight SCK pulses and returns a received byte. The same abstraction can be studied independently in SPI-01.
2. **SDHC command/response FSM.** Emits six-byte commands; receives R1/R3/R7, handles CS_n, bounded retries, timeouts, data-token handling and CRC16.
3. **Block RAM, 512 x 8 bits.** Retains the received sector with a registered synchronous peek_addr/peek_data access path.
4. **UART reporting FSM.** Emits SD PASS SIG=xxxx or SD ERR=xx and controls the six activity LEDs.

Core reference FSM states are S_IDLE, S_STARTUP, S_SEND, S_WAIT, S_DONE; protocol steps include PH_POWER, PH_GAP, PH_CMD, PH_R1, PH_EXTRA, PH_TOKEN, PH_DATA, PH_CRC_HI, PH_CRC_LO and PH_RELEASE.

RAM timing trap: the peek_data output is registered at posedge clk. A new peek_addr cannot be treated as a same-cycle combinational read. The wrapper deliberately waits in ST_PEEK0_WAIT and ST_PEEK1_WAIT before capturing the corresponding bytes.

6 Verilog: the real reference design

The full implementation is lab05_sd_init_read0.v. It is longer than a teaching snippet because it includes timeouts, response checking and disciplined sequencing. The following are **source-derived excerpts**:

```
reg [7:0] sector [0:511]
    /* synthesis syn_ramstyle = "block_ram" */;
```

```
always @(posedge clk)
    peek_data <= sector[peek_addr];

lab05_spi_byte #(.HALF_PERIOD(HALF_PERIOD)) u_spi (
    .clk(clk), .reset_n(reset_n),
    .start(spi_start), .tx_data(spi_tx),
    .miso(sd_miso), .sck(sd_sck), .mosi(sd_mosi),
    .busy(spi_busy), .done(spi_done), .rx_data(spi_rx)
);
```

The six command bytes are generated using logic equivalent to the reference `command_byte` function:

```
case (pos)
  3'd0: command_byte = 8'h40 | {2'b00,cmd};
  3'd1: command_byte = (cmd==6'd41) ? 8'h40 : 8'h00;
  3'd2: command_byte = 8'h00;
  3'd3: command_byte = (cmd==6'd8) ? 8'h01 : 8'h00;
  3'd4: command_byte = (cmd==6'd8) ? 8'haa : 8'h00;
  default: command_byte = (cmd==6'd0) ? 8'h95 :
                          (cmd==6'd8) ? 8'h87 : 8'h01;
endcase
```

In **this first experiment**, the LBA is fixed at zero; thus the CMD17 address bytes are zero. An SD-02 design must latch an arbitrary 32-bit LBA, preserve session state between successive reads and handle repeated requests. **That capability must not be attributed to SD-01.**

6.1 Data CRC16: checking the payload rather than counting bytes

The SD data-block CRC16 polynomial is

$$P(x) = x^{16} + x^{12} + x^5 + 1.$$

In the bitwise update routine shown here, the corresponding polynomial mask is `0x1021` and the initial remainder is zero. Each byte is processed over its eight bits:

```
function [15:0] crc16_next;
  input [15:0] previous;
  input [7:0] value;
  integer b;
  reg [15:0] c;
  begin
    c = previous;
    for (b=7; b>=0; b=b-1)
      if (c[15] ^ value[b])
        c = {c[14:0],1'b0} ^ 16'h1021;
      else
```

```
        c = {c[14:0],1'b0};
    crc16_next = c;
  end
endfunction
```

A successful CRC16 comparison establishes agreement between the received block and the checksum transmitted by the card; it does **not** establish filesystem correctness or the existence of meaningful user files.

7 SystemVerilog: a portable-controller design sketch

A future SystemVerilog version could use a typed protocol-phase enumeration and make data handling explicit. This is a **fragment**, not an operational replacement for SD-01:

```
typedef enum logic [3:0] {
  POWER, GAP, CMD, WAIT_R1, READ_EXTRA,
  WAIT_TOKEN, READ_DATA, CRC_HI, CRC_LO, RELEASE
} sd_phase_t;

sd_phase_t phase;
logic [8:0] data_index;
logic [15:0] crc_q;
logic [7:0] sector [0:511];
logic [7:0] spi_rx;
logic spi_done;

always_ff @(posedge clk) begin
  if (spi_done && phase == READ_DATA) begin
    sector[data_index] <= spi_rx;
    crc_q <= crc16_update(crc_q, spi_rx);
  end
end
```

Before this can be compiled as a full controller, it needs reset behavior, index bounds, `crc16_update`, a six-byte command sequencer, token handling, timeout logic and fully defined state transitions. The comparison criterion should be **identical accepted data and UART status** for the same SD model, not superficial source similarity.

A future SystemVerilog testbench may add assertions for chip-select transitions, memory bounds and transaction completion, and a configurable SDHC **bus functional model** (BFM) that returns both valid and intentionally malformed responses.

8 VHDL: the same protocol in a typed FSM

For a VHDL design, use `numeric_std`, constrained index ranges and explicit state types. Again, the following is a **declarative/process fragment**, not a compile-ready entity:

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

-- Architecture declaration fragments:
type sd_phase_t is (
  power_up, gap, send_cmd, wait_r1, read_extra,
  wait_token, read_data, crc_hi, crc_lo, release_bus
);
type sector_t is array (0 to 511)
  of std_logic_vector(7 downto 0);
signal sector     : sector_t;
signal data_index : integer range 0 to 511 := 0;
signal phase      : sd_phase_t := power_up;

-- Inside a clocked process, after validating the bounds:
if phase = read_data and spi_done = '1' then
  sector(data_index) <= spi_rx;
  -- crc_q <= crc16_update(crc_q, spi_rx);
end if;
```

A separate VHDL SPI master, SD command FSM, reset strategy and GHDL verification project must be written and tested before any equivalence or FPGA resource claim is made.

9 Testbenches: a simulated card and negative cases

The published `tb_lab05_sd_init_read0.v` is a behavioral SDHC SPI peripheral. It responds to commands and supplies a deterministic sector ending with bytes `0x55`, `0xAA` and a valid CRC16.

```systemverilog
// Excerpt: payload generation in the simulated SD card.
function [7:0] sector_byte;
  input integer addr;
  begin
    if (addr==0)        sector_byte=8'heb;
    else if (addr==1)   sector_byte=8'h58;
    else if (addr==2)   sector_byte=8'h90;
    else if (addr==510) sector_byte=8'h55;
    else if (addr==511) sector_byte=8'haa;
    else sector_byte=(addr & 255) ^ 8'ha5;
  end
endfunction
```

This checks a known-good protocol path. The following scenarios are **proposed extensions**, not a claim that every row is covered by the current CI:

Scenario	Required outcome
Valid R1 and data token FE	512 bytes and CRC PASS; bounded done
CMD8 returns wrong echo	Deterministic error; never false PASS
ACMD41 never becomes ready	Timeout and safe chip-select release
MISO stuck high	R1 or token timeout
Corrupted CRC	Error code; no success report
SDSC card identified	Explicit unsupported-card result or new tested variant
Reset mid-transfer	No card write and safe recovery on new initialization

9.1 SystemVerilog testbench outline

```
// Illustration: a real TB must also supply a card model and watchdog.
initial begin
  reset_n = 0;
  repeat (4) @(negedge clk);
  reset_n = 1;
  start = 1;
  @(negedge clk) start = 0;
  wait (done);
  assert (error_code == 8'h00)
    else $fatal(1, "SDHC failure: %h", error_code);
  $display("SD_SV_MODEL_PASS");
end
```

A full testbench **must bound** `wait(done)` with a finite timeout and compare actual sector bytes, CRC and state transitions. Simply printing the word PASS after a wait is not enough.

9.2 VHDL testbench outline

```
-- Checks within a VHDL testbench after bounded completion:
assert error_code = x"00"
  report "SDHC status error" severity failure;
assert sector_510 = x"55" and sector_511 = x"AA"
  report "Unexpected final sector bytes" severity failure;
report "SD_VHDL_MODEL_PASS" severity note;
```

A complete VHDL test requires clock generation, an SPI response BFM, a finite watchdog, receipt of all 512 bytes and CRC comparison.

10 External scripts, synthesis, and SRAM-only loading

To reproduce **the published Verilog project** using PowerShell 7, Icarus Verilog and Gowin EDA:

```
git clone https://github.com/AIDevelopersMonster/Sipeed-FPGA.git
cd Sipeed-FPGA

# Independent RTL testbenches, wrapped by a script:
& .\boards\tang-nano-9k\experiments\sd-sector0\test.ps1

# Separate Gowin build; NEVER automatically programs the board:
& .\boards\tang-nano-9k\experiments\sd-sector0\build.ps1
```

`build.ps1` stages a distinct compilation, records `input_sha256.csv`, leaves a Gowin log and reports, and generates a candidate named `sd_sector0_nano9k.fs`. **It does not write configuration Flash or SRAM.** Always review synthesis, place-and-route, IO constraints, setup/hold timing and the artifact hash before any programming. The presence of `.fs` by itself is not a hardware acceptance result.

10.1 Manually program *volatile SRAM* after checking reports

```
# Use the exact absolute path printed as SD01_BITSTREAM_CANDIDATE.
$fs = 'C:\<path-from-your-Gowin-report>\sd_sector0_nano9k.fs'
$prog = 'C:\Gowin\Gowin_V1.9.9Beta-
 ↪  4_Education\Programmer\bin\programmer_cli.exe'

if (-not (Test-Path -LiteralPath $fs)) { throw 'Bitstream absent' }
Get-FileHash -LiteralPath $fs -Algorithm SHA256
& $prog --device GW1NR-9C --operation_index 2 --fsFile $fs --cable-index 1
if ($LASTEXITCODE -ne 0) { throw 'SRAM programming failed' }
```

PowerShell note: the programmer command is deliberately on **one line**. If you split it, use PowerShell continuation (backtick), not a Unix backslash. Never use a Flash programming operation for this first experiment.

Then open a terminal at **115200 baud, 8N1**. Insert the card before power-up and press **physical S2** to rerun the test if necessary. For cards whose sector 0 ends with 55 AA, the UART reports `SD PASS SIG=55AA`; the signature bytes can legitimately differ on other media.

10.2 Error codes specific to this Verilog design

Code	Meaning
E1	CMD0 response error
E2	CMD8 response error
E3	CMD55 response error
E4	ACMD41 timeout
E5	CMD58 response error
E6	CMD17 response error
E7	No confirmed SDHC high-capacity mode

Code	Meaning
E8	R1 response timeout
E9	Data-token error or timeout
EA	Received data CRC16 mismatch
EB	CMD8 echo mismatch

These are **project-specific diagnostic codes**, not a universal SD Association error vocabulary.

11 From a first sector to a filesystem: research program

As a PiKoSoL milestone, the first SD card probe may later be superseded. As a teaching experiment it remains useful **independently of the future SoC architecture**. Every positive or negative follow-up should freeze its code, conditions and result.

Follow-up	Research question	Minimum acceptance
SD-02: multi-LBA	Can an FPGA read arbitrary blocks without re-init?	Three nonconsecutive LBAs, CRC PASS
SD-03: partition structures	MBR or superfloppy volume?	Locate boot sector using validated fields; not 55AA alone
SD-04: FAT32	Can RTL traverse FAT cluster chains?	Correct `DIR`, then TYPE of a prepared text file
SD-05: architecture comparison	Pure FSM vs PicoRV32 firmware vs hybrid?	LUT/FF/BSRAM, latency and error behavior under fixed conditions
SD-06: portability	Does the design transfer beyond Nano 9K?	Separate PnR, STA and real-board PASS

Negative results are scientific observations. A timing failure, unexpected SDHC response, resource overflow, speed limit or difficult state-machine corner case should be preserved with its source revision and evidence, even when the PiKoSoL system later switches to another approach.

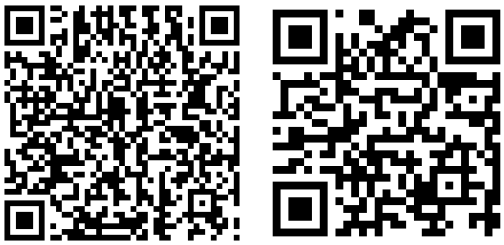
11.1 Physical experiment record template

Field	Fill in after running the experiment
Date, exact board / PCB revision	…
Git commit and bitstream SHA256	…
Card model, family and capacity	…
Gowin / Icarus versions and OS	…

Field	Fill in after running the experiment
LUT / FF / BSRAM; STA setup / hold	...
Complete UART output	...
SIG or ERR received	...
What the measurement actually establishes	...
Verdict	PASS / FAIL / NOT TESTED

12 References, code, and video demonstration

- **SD-01: RTL, testbenches, constraints and build:** https://github.com/AIDevelopersMonster/Sipeed-FPGA/tree/main/boards/tang-nano-9k/experiments/sd-sector0
- **SPI-01: the preceding learning step:** https://github.com/AIDevelopersMonster/Sipeed-FPGA/tree/main/examples/peripherals/spi/01-mode0-byte
- **RTL CI run for the fixed published revision:** https://github.com/AIDevelopersMonster/Sipeed-FPGA/actions/runs/40052632010
- **Original hardware demonstration video:** https://youtu.be/ybEUMzt7Gqg
- **SD Association simplified specifications:** https://www.sdcard.org/downloads/pls/
- **Microchip SPI modes and timing:** https://onlinedocs.microchip.com/oxy/GUID-6F1B86DC-C230-43D1-A405-0CC2E7EDF7EE-en-US-2/GUID-EAC2D943-F395-4D47-98C8-4EEBC798BFA2.html



Edition 0.1 boundary: An extensible technical educational foundation. A complete SV/VHDL controller, negative-response test suite, independent PnR/STA + board acceptance record for the published snapshot, and FAT32 require separate implementations and checks.